to label the sentiment associated with a sentence, we could label it as positive, negative or neutral. On the other hand, problems where we have to predict whether a fruit is an apple or an orange will have binary labels. Table 1 gives a sample data set for a classification problem.

In this table, the value of the last column, i.e., loan approved, is expected to be predicted based on the other variables. In the subsequent sections, we will learn how to train and evaluate a classifier using Python.

Training and evaluating a classifier

In order to train a classifier, we need to have a data set containing labelled examples. Though the process of cleaning the data is not covered in this section, it is recommended that you read about various data preprocessing and cleaning techniques before feeding your data set to a classifier. In order to process the data set in Python, we will import the pandas package and the data frame structure. You may then choose from a variety of classification algorithms such as decision tree, support vector classifier, random forest, XG boost, ADA boost, etc. We will look at the random forest classifier, which is an ensemble classifier formed using multiple decision trees.

```
from sklearn.ensemble import
RandomForestClassifier
from sklearn import metrics

classifier = RandomForestClassifier()

#creating a train-test split with a
proportion of 70:30
X_train, X_test, y_train, y_test =
train_test_split(X, y, test_size=0.33)

classifier.fit(X_train, y_train) #train
the classifier on the training set

y_pred = classifier.predict(X_test)
#evaluate the classifier on unknown data

print("Accuracy: ", metrics.accuracy_
score(y_test, y_pred)) #compare the
predictions with the actual values in
the test set
```

Although this program uses accuracy as the performance metric, a combination of metrics should be used, as accuracy tends to generate non-representative results when the test set is imbalanced. For instance, we will get a high accuracy if the model gives the same prediction for every record and the data set that is used to test the model is imbalanced, i.e., most of the records in the data set have the same class that the model predicted.

Tuning a classifier

Tuning refers to the process of modifying the values of the hyperparameters of a model in order to improve its performance. A hyperparameter is a parameter whose value can be changed to improve the learning process of the algorithm.

The following code depicts random search hyperparameter tuning. In this, we define a search space from which the algorithm will pick different values, and choose the one that produces the best results:

```
from sklearn.model_selection import
RandomizedSearchCV

#define the search space
min_samples_split = [2, 5, 10]
min_samples_leaf = [1, 2, 4]
grid = {'min_samples_split' : min_
samples_split, 'min_samples_leaf' :
min_samples_leaf}

classifier =
RandomizedSearchCV(classifier, grid,
n_iter = 100)

#n_iter represents the number of
samples to extract from the search
space
#result.best_score and result.best_
params_ can be used to obtain the best
performance of the model, and the best
values of the parameters

classifier.fit(X_train, y_train)
```

Voting classifier

You can also use multiple classifiers and their predictions to create a model that will give a single prediction based on the individual predictions. This process (in which only the number of classifiers that voted for each prediction is considered) is called hard voting. Soft voting is a process in which each classifier generates a probability of a given record belonging to a particular class, and the voting classifier generates as its prediction, the class that obtained the maximum probability.

A code snippet for creating a soft voting classifier is given below:

```
soft_voting_clf = VotingClassifier(
        estimators=[('rf', rf_clf),
('ada', ada_clf), ('xgb', xgb_clf),
('et', et_clf), ('gb', gb_clf)],
        voting='soft')
    soft_voting_clf.fit(X_train, y_
train)
```

This article has summarised the use of classifiers, tuning a classifier and the process of combining the results of multiple classifiers. Do use this as a reference point and explore each area in detail. END

Reference

Jordan, M.I., & Mitchell, T.M. (2015), Machine learning: Trends, perspectives, and prospects. Science, 349(6245), 255-260.

By: Gayatri Venugopal

The author is a faculty member at a private university, and is currently pursuing her PhD in the areas of machine learning and natural language processing.